Informe de Auditoría y Bastionado de Seguridad (Hardening) — 404Shop

Bitácora de Hardening - 404Shop

Auditor: Michael Washington Ortega Hernandez
Fecha de inicio: 1 octubre

fecha de finalizacion octubre 8

Objetivo: Identificación de vulnerabilidades, refactorización de arquitectura, mitigación de riesgos de fuga de información e implementación de un modelo de defensa en profundidad (*Defense in Depth*) en una aplicación e-commerce full-stack

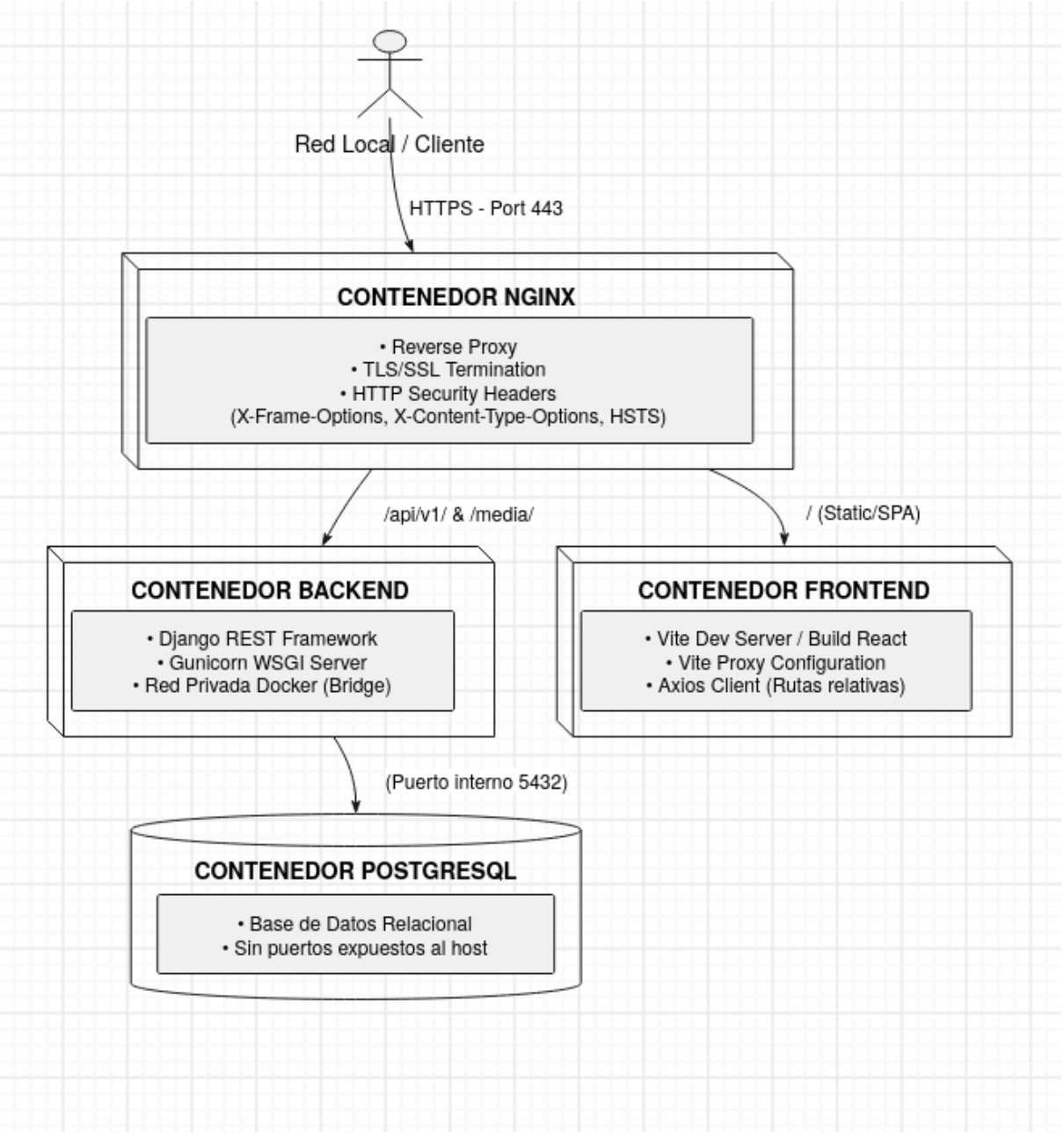
1. Resumen Ejecutivo

El proyecto **404Shop** presentaba en su estado inicial múltiples deficiencias de seguridad comunes en entornos de desarrollo: credenciales hardcodeadas en repositorios públicos, ausencia de cifrado en tránsito (HTTP claro), sobreexposición de metadatos internos en la API REST (DTOs no sanitizados) y exposición directa de servicios críticos (PostgreSQL y Gunicorn) a la red local.

A través de un proceso de bastionado iterativo, se logró:

- **Aislar completamente la capa de datos** en redes internas no enrutables desde el host.
- **Implementar cifrado estricto TLS/SSL** (HTTPS con TLSv1.2 y TLSv1.3).
- **Reducir la superficie de ataque en la API REST** mediante la restructuración de serializadores en Django REST Framework.
- **Eliminar la fuga de información por depuración** (`DEBUG = False`).
- **Verificar la postura de seguridad mediante análisis dinámico (DAST)** con OWASP ZAP y análisis de paquetes con Wireshark, obteniendo cero (0) vulnerabilidades críticas o altas.

2. Arquitectura del sistema



Fase 1: reconocimiento, despliegue e identificacion inicial

Primer paso antes de endurecer el sistema, obviamente inicializarlo, montar el entorno para ver como funciona, cacharrearlo y comprenderlo

Despliegue

Hallazgo de credenciales hardcodeadas y falta de .env

- **.env faltante:**Al intentar levantar el entorno con docker (backend y postgreSQL) el proceso falló por la ausencia de el archivo .env en el backend, se a ciencia cierta que subir el .env a un repositorio es algo que no se deberia hacer, sin embargo es buena practica proporcionar un .env.example ni documentó las variables de entorno necesarias, lo que rompe el despliegue para los nuevos desarrolladores

  Solucion: revisando el codigo en busca de alguna pista o señal de que variables debia usar para levantar el backend, me encontre que el autor original dejó las credenciales de la base de datos escritas en texto plano (hardcodeadas) en un archivo que se sube al repositorio publico (hardcoded secrets), una vulnerabilidad critica. gracias a este archivo, aunque no haya un .env ni un .env.example, puedo saber que poner en el .env que voy a creae para que el backend se conecte a la base de datos.

  .env

  ```
  SECRET_KEY=django-insecure-clave_de_prueba_para_local_123
  DEBUG=True
  DB_NAME=tienda_db
  DB_USER=postgres
  DB_PASSWORD=admin
  DB_HOST=db
  DB_PORT=5432
  ```

- **Paquete ausente:** Una vez creado .env dentro de /backend/, levanté el entorno con docker-compose.yml, pero me encontré con otro problema, Django estaba intentando cargar el tema de administracion jazzmin, pero el paquete no estaba instalado dentro de el contenedor docker

  Solucion: en el archivo requeriments.txt agregue la dependencia

  ```
  django-jazzmin
  ```

- **DB vacia:** luego de crear el .env e implementar django-jazzmin en los pasos anteriores, el entorno se levantó pero con una advertencia importante

```
You have 21 unapplied migration(s). Your project may not
work properly...
```

no hay tablas necesarias para guardar nada

Solucion: Dejando corriendo el servidor, en otra consola ejecuté

```
docker compose exec backend python manage.py migrate
```

y me cree un usuario administrador para mi

```
docker compose exec backend python manage.py createsuper
user
```

Una vez hecho estos pasos para levantar el backend, levanto el frontend con

```
cd frontend
npm install
npm run dev
```

recibi un error

```
            cd frontend
      npm install
      npm run dev
node: error while loading shared libraries: libsimdjson.
so.33: cannot open shared object file: No such file or d
irectory
node: error while loading shared libraries: libsimdjson.
so.33: cannot open shared object file: No such file or d
irectory
```

Al estar trabajando en un entorno Arch linux, simplemente debia reinstalar o sincronizar dependencias, luego de unos pasos extra irrelevantes para la documentacion, el entorno se levantó

Nota: En my dispositivo host (CachyOS/Arch Linux), se solucionó este desajuste de bibliotecas compartidas, sincronizando dependencias del

sistema antes de levantar el SPA (Single Page Aplication)

Reconocimiento:

La pagina principal de la pagina web no tiene productos, obviamente porque no hay productos publicados, las principales paginas se desplegaron en http://localhost:5173/ y http://localhost:8000/

para acceder al panel admin http://localhost:8000/admin/, accedo con las credenciales que me cree de admin y agrego productos de prueba con cierto stock

- Codigo reciclado: la pagina parece estar diseñada originalmente para ropa, o parece que el desarrollador original reciclo codigo, no permitiendo quitar este apartado en otros productos, aunque es un detalle menor para el hardening, demuestra que es probable que no se haya saneado correctamente las entradas de datos en el backend
- **Fuzzing (envio de datos invalidos o aleatorios en un programa para ver si falla):** al introducir caracteres aleatorios, no se rompe nada, la barra de busquda funciona perfectamente y no se ropme ni muestra un erorr Django o PostgreSQL

Sobeexposicion de datos en al API REST:

Auditorias de seguridad y optimizacion de Endpoints (DTOs)

Mientras estaba en la fase de integración, entré el backend en Django REST Framework y el frontend, se realizo una auditoria de las cargas utiles (payloads) utilizando las herramientas de desarrollo del navegador

1. **Hallazgo de sobreexposicion de datos:** Se detectó que los endpoints de la API (/api/V1/productos/) devolvian entidades completas de la base de datos sin filtrar. Incluidos metadatos de auditoria interna innecesarios para el cliente, como por ejemplo
    - Marcas de tiempo de creacion y modificacion (created_at, updated_at)
    - Exposicion detallada del inventario de variantes
2. **Justificacion del Riesgo:** Exponer esta infromacion incrementa la superficie de ataque al revelar la estructura de los modelos de datos y facilita la recoleccion de inteligencia comercial automatizada sobre el stock de negocio

3. **Plan de mitigacion:** planeo implementar el uso estricto de Serializers personalizados en Django para asegurar que la API solo transmita atributos necesarios para el funcionamiento de las vistas de usuario

Fase 2: Pruebas de penetración y búsqueda de vulnerabilidad

Una vez la pagina web desplegada, con backend y frontend funcionando y con ciertos productos agregados y con funcionalidad verificada, llega las pruebas de pentesting y que tan vulnerable esta este sitio.

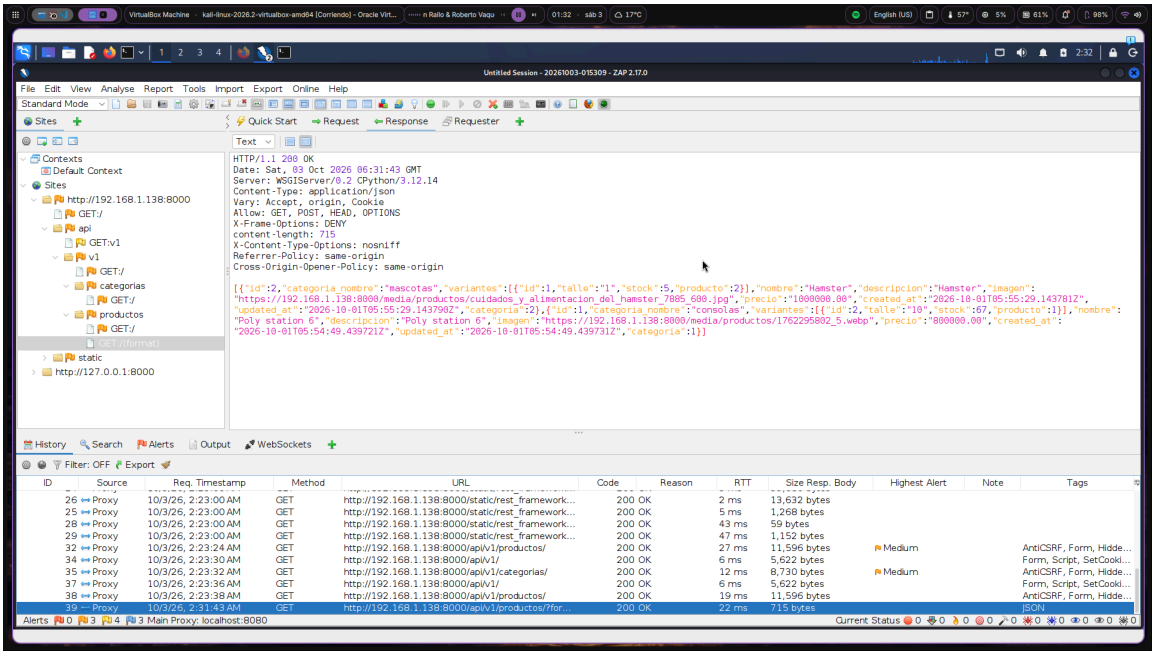
Realice diferentes pruebas de penetracion sencillas sobre la aplicacion y la infraestructura para evaluar el alcance maximo de compromiso de datos e infraestructura anfitriona

Vector de ataque evaluado	Herramienta / técnica	Resultado	Diagnostico
Inyeccion SQL(SQLi)	Payloads SQL en formulatios y QUery Params	No encontré vulnerabilidad	
Ejecución remota de codigo (RCE)	Fuzzing de parámetros	No encontré o no logré vulnerar	Sin uso de funciones del SO expuestas
Escape de contenedor	Inspeccion de cgroups y montaje	Contenido	Aislamiento efecito, no hay sockets del docker montado dentro de contenedores
Interceptación de tráfico	Wireshark / OWASP ZAP	Encontré una gran vulnerabilidad	Tráfico HTTP en texto plano, credenciales transmitidas sin cifrar

1. Empezando por que el sitio no tiene ningún tipo de estacionario, todos los datos viajan en texto plano, es un http, para demostrar lo inseguro que es esto en un entorno real, en kali linux con owasp zap estoy sacando como todos los DTOs están sobre expuestos para un atacante en la misma red wifi, el cual podría robar esa información al no existir ningún tipo de cifrado.

a. El settings.py no tine autorizado otras IPs que no sean el localhost, por motivos practicos y realistas para simular que el entorno local es un servidor publico en internet, voy a introducir mi ip dinamica en IPs autorizadas

```
ALLOWED_HOSTS = ["localhost","127.0.0.1", "192.168.1.13
8"]
```




```
HTTP/1.1 200 OK
Date: Sat, 03 Oct 2026 06:31:43 GMT
Server: WSGIServer/0.2 CPython/3.12.14
Content-Type: application/json
Vary: Accept, origin, Cookie
Allow: GET, POST, HEAD, OPTIONS
X-Frame-Options: DENY
content-length: 715
X-Content-Type-Options: nosniff
Referrer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin

[{"id":2,"categoria_nombre":"mascotas","variantes":[{"id":
1,"talle":"1","stock":5,"producto":2}],"nombre":"Hamste
r","descripcion":"Hamster","imagen":"https://192.168.1.138:
```

8000/media/productos/cuidados_y_alimentacion_del_hamster_7885_600.jpg","precio":"1000000.00","created_at":"2026-10-01T05:55:29.143781Z","updated_at":"2026-10-01T05:55:29.143790Z","categoria":2},{"id":1,"categoria_nombre":"consolas","variantes":[{"id":2,"talle":"10","stock":67,"producto":1}],"nombre":"Poly station 6","descripcion":"Poly station 6","imagen":"https://192.168.1.138:8000/media/productos/1762295802_5.webp","precio":"800000.00","created_at":"2026-10-01T05:54:49.439721Z","updated_at":"2026-10-01T05:54:49.439731Z","categoria":1}]

Aqui se evidencia como se esta escupiendo la estructura completa de la base de datos en formato JSON, incluyendo los campor internos de auditoria (created_at, updated_at) y el inventario desglosdo (stock) para los productos

Detalle de Pruebas de Limites e Impacto (Pruebas Negativas)

1. Evaluación de Manipulación Directa de Base de Datos (SQLi)

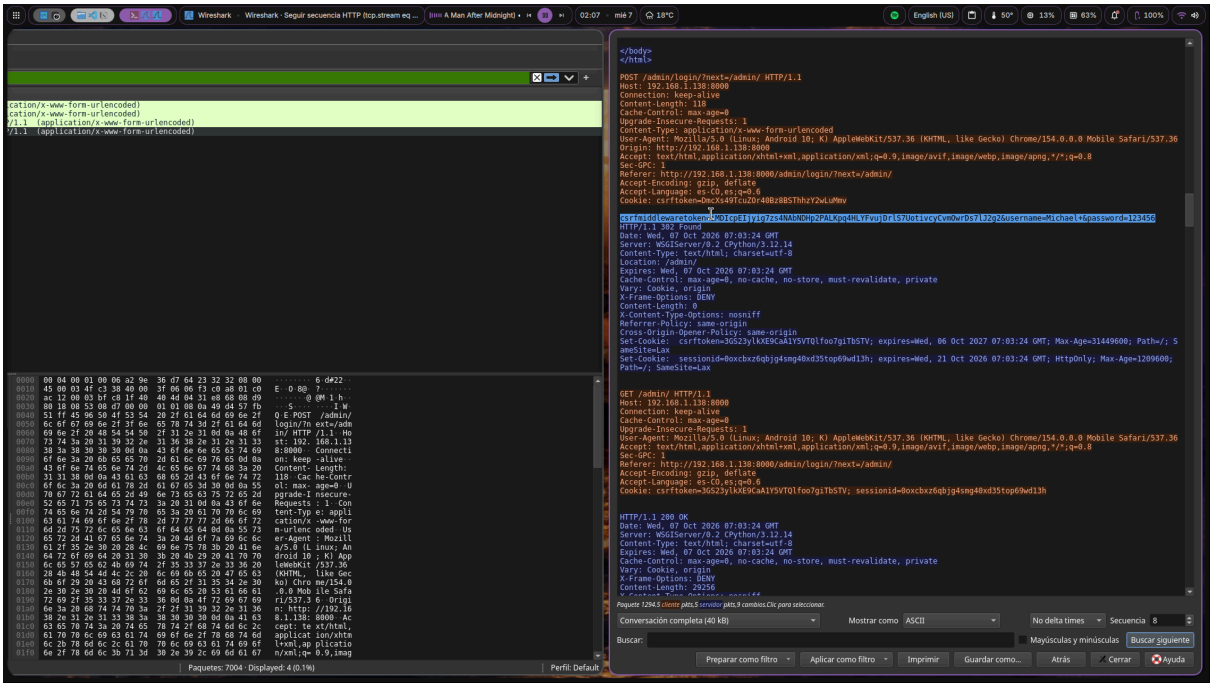
- **Objetivo:** Intentar alterar, saltar autenticaciones o volcar registros de la base de datos SQLite/PostgreSQL.
- **Resultado:** Se enviaron payloads estándar de prueba en los campos de usuario y contraseña. El marco de trabajo procesó las entradas como literales de cadena de texto sin interpretar comandos SQL.
- **Conclusión:** La capa de persistencia se encuentra protegida por el diseño de Django ORM.

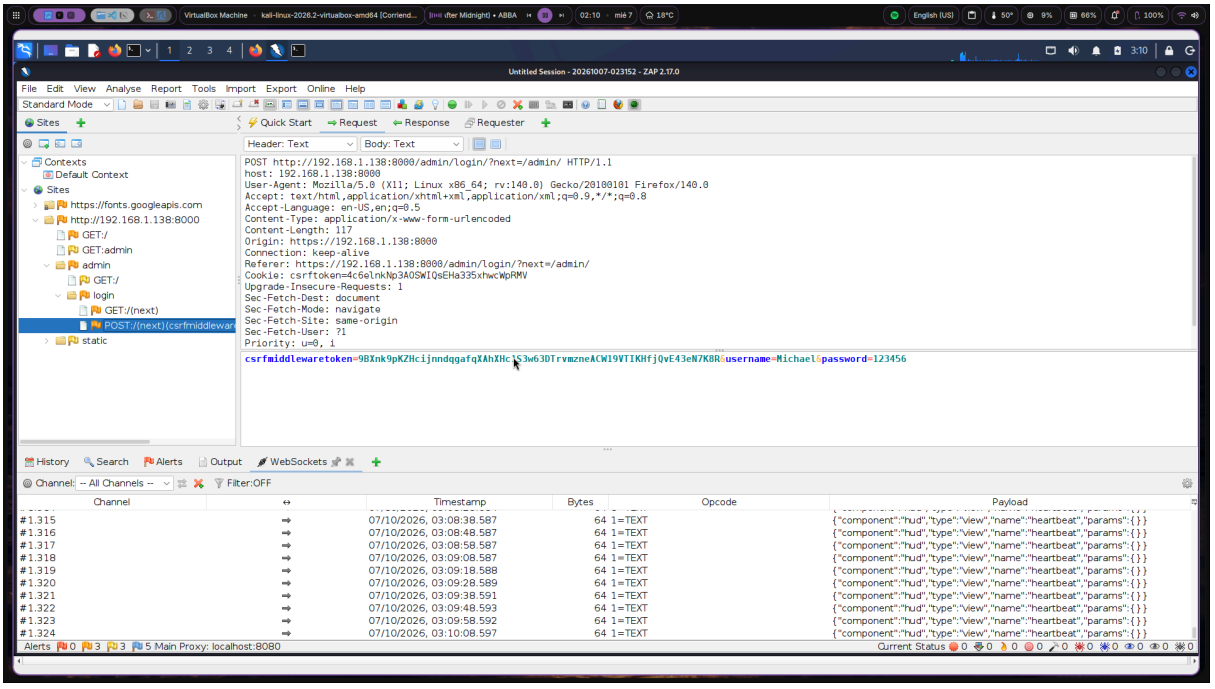
2. Evaluación de Escalamiento al Sistema Anfitrión (Host Escape)

- **Objetivo:** Evaluar si un compromiso del servicio web permitiría modificar el sistema operativo base (CachyOS en este caso)
- **Resultado:** La aplicación se encuentra confinada dentro del entorno virtualizado de Docker. No existen montajes directos del socket de Docker (`/var/run/docker.sock`) ni volúmenes del sistema de archivos raíz (`/`) expuestos al contenedor.
- **Conclusión:** El alcance potencial de un ataque a nivel de aplicación queda acotado exclusivamente al perímetro del contenedor, garantizando la

integridad del sistema operativo host.

Fase 3: hallazgos / vulnerabilidades encontradas





sniffing de red multidispositivos (wireshark)

filtro aplicado `http.request.method == "POST"`

analice el analisis de flujo TCP/HTTP del paquete de autenticacion

que pasó en este paso, muy grave, filtracion de contraseñas y datos sensibles en texto plano al transferirse, en este punto abrí wireshark y estuve escuchando mi red, desde mi celular entre a la pagina 404shop e inicie sesion con las credenciales de usuario que cree anteriormente y wireshark pudo interceptar esa transferencia, confirmando que las credenciales van por texto plano antes de guardarse encriptadas en la base de datos, ineficiente

Fase 4: Endurecimiento

Eliminar las credenciales escritas en texto plano (hardcoded secrets), configurando la lectura de variables de entorno mediante .env y activar configs de seguridad en Django

1. Crear .en.example

```
SECRET_KEY=cambiar_por_una_clave_segura_en_produccion
DEBUG=False
ALLOWED_HOSTS=localhost,127.0.0.1
DB_NAME=tienda_db
DB_USER=postgres
DB_PASSWORD=contrasena_segura_aqui
DB_HOST=db
DB_PORT=5432
```

2. Generar una SECRET_KEY segura para no usar contraseñas en texto plano

```
python3 -c 'import secrets; print(secrets.token_urlsafe
(50))'
```

```
SECRET_KEY= SECRET_KEY
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1,192.168.1.138
DB_NAME=tienda_db
DB_USER=postgres
DB_PASSWORD=admin
DB_HOST=db
DB_PORT=5432
```

3. Actualizar backend/settings.py

```
# Reconocer el protocolo HTTPS cuando venga de un proxy
inverso
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'ht
tps')

##Forzar flags 'Secure' en cookies de sesión y CSRF
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True

# Cabeceras de proteccion del navegador contra XSS y Cli
ckjacking
SECURE_BROWSER_XSS_FILTER = True
SECURE_CONTENT_TYPE_NOSNIFF = True
X_FRAME_OPTIONS = 'DENY'
```

4. Para evitar filtracion de datos innecesarios o DTOs, lo filtramos en los serializadores, en serializers.py, puedo ver como se esta imprimiento todo

```
class CategoriaSerializer(serializers.ModelSerializer):
    class Meta:
        model = Categoria
        fields = "__all__"


class VarianteProductoSerializer(serializers.ModelSerial
izer):
    class Meta:
        model = VarianteProducto
        fields = "__all__"


class ProductoSerializer(serializers.ModelSerializer):
    categoria_nombre = serializers.CharField(
        source="categoria.nombre",
        read_only=True
    )

    variantes = VarianteProductoSerializer(
```

```
        many=True,
        read_only=True
    )

    class Meta:
        model = Producto
        fields = "__all__"
```

Asi que en las vistas consumidas por usuarios/frontend escojo lo que si quiero que se muestre

```
class CategoriaSerializer(serializers.ModelSerializer):
    class Meta:
        model = Categoria
        fields = ["id", "nombre", "descripcion", "posici
on", "activo"]


class VarianteProductoSerializer(serializers.ModelSerial
izer):
    class Meta:
        model = VarianteProducto
        fields = ["id","talle", "color", "precio", "stoc
k", "producto"]


class ProductoSerializer(serializers.ModelSerializer):
    categoria_nombre = serializers.CharField(
        source="categoria.nombre",
        read_only=True
    )

    variantes = VarianteProductoSerializer(
        many=True,
        read_only=True
    )

    class Meta:
```

```
        model = Producto
        fields = "            "id",
            "titulo",
            "descripcion",
            "imagen",
            "posicion",
            "activo",


class AnuncioSerializer(serializers.ModelSerializer):
    class Meta:
        model = Anuncio
        fields = [
            "id",
            "titulo",
            "descripcion",
            "imagen",
            "posicion",
            "activo",
            "fecha_inicio",
            "fecha_fin"
        ]
```

CIfrado TSL/SSL y reverse proxy con Nginx

1. Creacion de certificado SSL/TSL y configuracion de Nginx como proxy inverso para eliminar el trafico de texto plano

```
mkdir -p nginx/certs
```

genero los certificados para la direccion ip de prueba de entorno local, restingiendo la comunicacion no cifrada

```
# Creación del directorio seguro para certificados
mkdir -p nginx/certs

# Generación de clave privada y certificado X.509
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
```

```
  -keyout nginx/certs/selfsigned.key \
  -out nginx/certs/selfsigned.crt \
  -subj "/CN=192.168.1.138"
```

Configuración del proxy inverso, para todo trafico entrante por el puerto 80, se redirige a 301, solo restricción exclusiva a protocolos seguros TSL1.2 y TSL1.3, deshabilitando versiones obsoletas e inseguras

```
server {
    listen 80;
    server_name 192.168.1.138 localhost;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name 192.168.1.138 localhost;

    ssl_certificate /etc/nginx/certs/selfsigned.crt;
    ssl_certificate_key /etc/nginx/certs/selfsigned.key;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    add_header Strict-Transport-Security "max-age=31536000;
includeSubDomains" always;
    add_header X-Frame-Options "DENY" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;

    location / {
        proxy_pass http://backend:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwa
rded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
```

```
    }
}
```

asi, garantizo la confidencialidad e integridad del trafico entre clientes y el servidor backend

ademas de resistencia a sniffing, cualquier captura de paquetes realizada en la red unicamente registrara tramas cifradas mediante TSL, haciendo que no se puedan leer credenciales o payloads en texto plano

5. Aplicacion de principio de privilegio de red y segmentacion de red (Defense in depth)

En este paso voy a darle funcionalidad al reverse proxy para aislar y asegurar el website, aisló la base de datos en una red interna para que ningún puerto quede expuesto en mi maquina fisica, tampoco expongo el puerto 8000 de mi maquina y del backend al exterior, solo Nginx podra hablar con el y nginx sera el unico visible hacia afuera, y es el que gestionara los puertos 80 y 443

Sustiyuto el servidor de desarrollo Django por Gunicorn

en el docker-compose.yml las variables de env estan escritas en texto plano

```
    environment:
      POSTGRES_DB: tienda_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: admin
```

esto es peligrosisimo, asi que lo cambio por variable sy que el propio docker compose los busque desde el archivo .env, lo mismo con el puerto, a pesar de que el puerto 5432 es el puerto comun de PostgreSQL, tenerlo expuesto y que cualquiera lo pueda ver es delicado

```
    ports:
      - "5432:5432"
```

asi que esto es lo que hice

```
  db:
    image: postgres:15
    container_name: postgres_db
```

```
    restart: always
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

ahora la base de datos no esta expuesta al mundo, sino que se comunica por el puerto 8000 con nginx

```
version: '3.8'

services:
 # Base de datos aislada
  db:
    image: postgres:15
    container_name: postgres_db
    restart: always
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - backend_network


# Backend (Servidor WSGI/Django pero ahora con Gunicorn)
  backend:
    build:
      context: ./backend
    container_name: django_backend
    restart: always
    command: gunicorn config.wsgi:application --bind 0.0.0.
0:8000
    volumes:
```

```
      - static_volume:/app/staticfiles
      - media_volume:/app/media
    env_file:
      - ./backend/.env
    depends_on:
      - db
    networks:
      - backend_network
      - frontend_network

# Proxy inverso y terminador ssl (Nginx)
  nginx:
    image: nginx:alpine
    container_name: 404shop_proxy
    restart: always
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:r
o
      - ./nginx/certs:/etc/nginx/certs:ro
      - static_volume:/app/staticfiles:ro
      - media_volume:/app/media:ro
    depends_on:
      - backend
    networks:
      -frontend_network

# Segmentacion o aislamiento de redes
networks:
  backend_network:
    internal: true
  frontend_network:

volumes:
  postgres_data:
  static_volume:
```

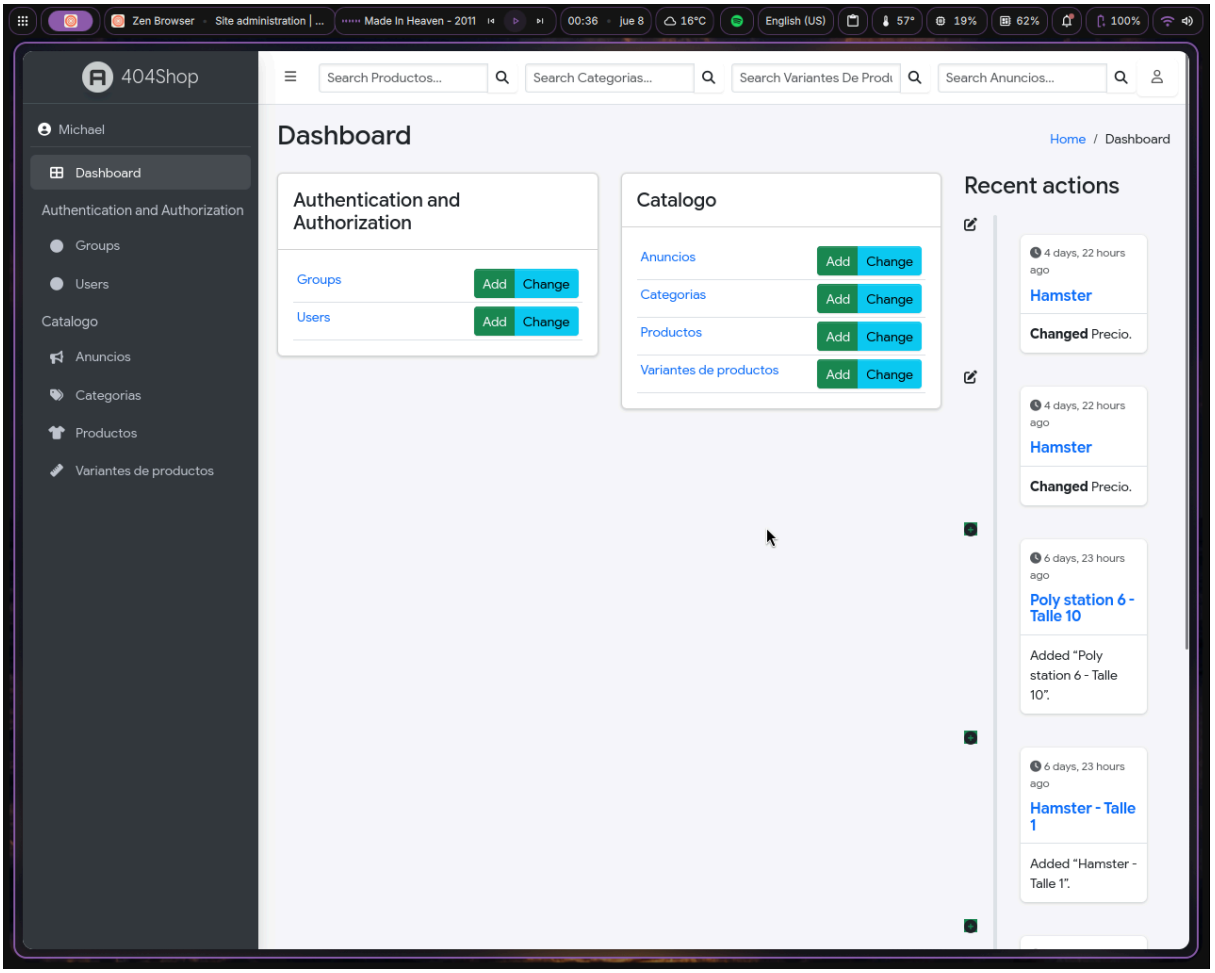
```
  media_volume:
  certs_volume:
```

ahora si, levanto el entorno para ver si no hubo errores de sintaxis

Luego de asegurar que todo estuviera bien, levante el proceso con la base de datos y backend, levantandose exitosamente, cree y actualice las tablas dentro del postgreSQL y junte todo el css, javascript e imagenes del panel de administracion en la carpeta staticfiles para que se entregen directamente al mavegador sin sobrecargar Django

```
docker compose exec backend python manage.py migrate
docker compose exec backend python manage.py collectstatic
--noinput
```

levante el frontend y al poner mi ip dinamica como direccion, efectivamente me redirecciona a https://... en vez de http://..., al ser un certificado autofirmado, me da una advertencia pero puedo continuar y todo esta en orden



Fase 5: Pruebas de seguridad para comprobar que el endurecimiento

Luego de todos los cambios, voy a presentar el documento detalladamente con las soluciones de fallos, la optimizancion de flujo de datos entre frontend y backend, la limplementacion de controles bastionados (Hardening) de infraestructura en el contenedor web. La meta principal fue garantizar la continuidad operativa del catalogo de productos y mitigar vectores en riesgo comunes como exposicion de trazas de codigo, muga de metadatos de la API y accesos no autorizados a servicios internos

Matriz de resolución de incidencias

Problema detectado	Causa	Solucion aplicada
Error 500 en backend	Desajuste entre modelos DB y campos en el MOdelSerializer, y el codigo en memoria de gunicorn desactualizado	Correcion de relaciones en serializers.py, ajuste de CORS_ALLOWED_ORIGINS y reinicio con rebuild de contenedores
Error de red / CORS	URLs absolutas con IP locales/ localhost en peticiones de Axios provocando bloqueos de seguridad por el origen cruzado	Sustitucion de domunios absolutos por rutas relativas
Estaso 200 ok pero devolviendo html	Peticiones relativas dirigidas a vite que al no reconocer las rutas relativas, devolvia la SPA de React	COnfiguracion del procy inverso en vite.config.js apuntando las rutas relativas hacia el backend
Pantalla en blanco / Excepcion JavaScript	intento de invocar .map() o filter() sobre respuestas no vectoriales o valores nulos	Programacion defensiva en componentes de React

Auditoria y registro de pruebas de bastionado (Hardening PoC)

1. Ocultamiento de informacion de errores y trazas (DEBUG = False)

Ahora la respuesta devuelve una pagina 404 HTML/JSON estandar limpia, eliminando por ocompleto detalles del URLconf y rutas de la API

```
curl -k -i https://192.168.1.138/api/v1/ruta-inexistente
-test/

HTTP/1.1 404 Not Found
Server: nginx/1.31.6
Date: Thu, 08 Oct 2026 06:52:57 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 179
Connection: keep-alive
X-Frame-Options: DENY
Vary: origin
X-Content-Type-Options: nosniff
Referrer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin
Strict-Transport-Security: max-age=31536000; includeSubD
omains
X-XSS-Protection: 1; mode=block


<!doctype html>
<html lang="en">
<head>
  <title>Not Found</title>
</head>
<body>
  <h1>Not Found</h1><p>The requested resource was not fo
und on this server.</p>
</body>
</html>
```

2. Aislamiento de capa de datos y servicios internos

   Ya no hay conexión directa no autorizada al postgreSQL (5432) o bypass del proxy inverso atacando Guinicorn (8000)

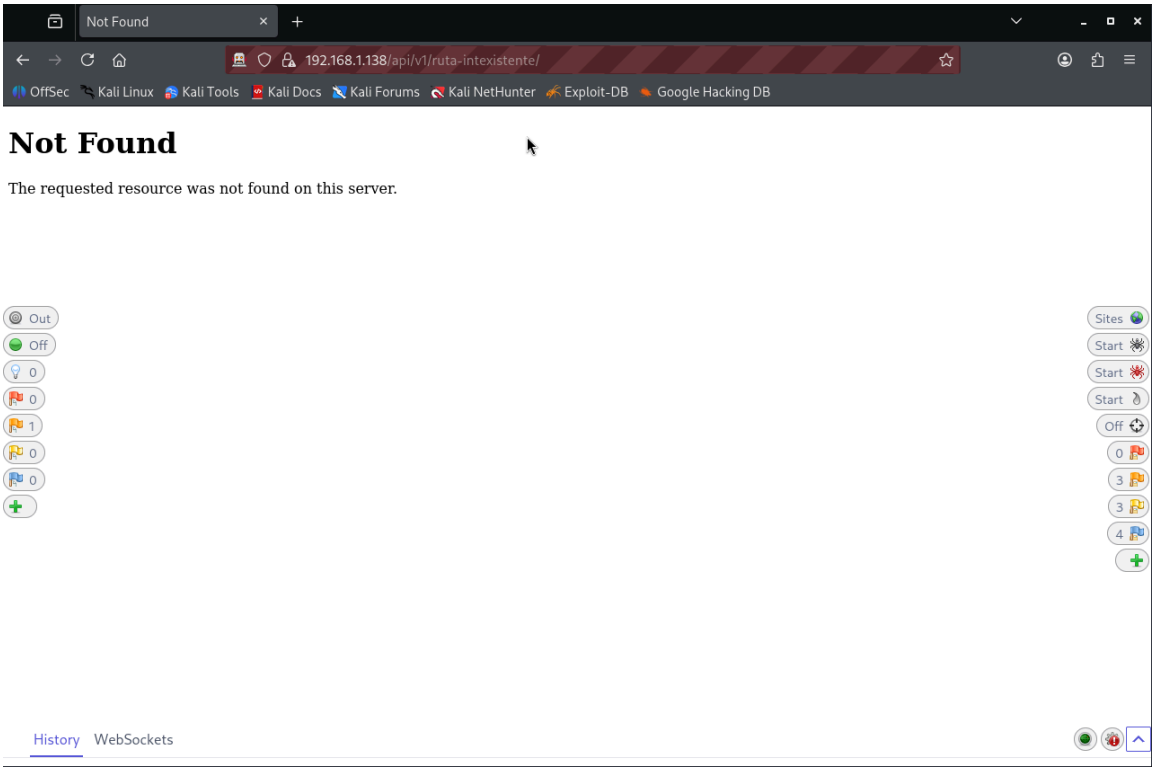
   Eliminé los bloques ports: en el servicio de PostgreSQL en el docker-comose-yml para mantener la comunicacion exclusivamente en la red

interna bridge del docker

```
nc -zv 192.168.1.138 5432 8000
nc: connect to 192.168.1.138 port 5432 (tcp) failed: Con
nection refused
nc: connect to 192.168.1.138 port 8000 (tcp) failed: Con
nection refused
```

3. Prueba de fuga de datos

   petición con error forzado



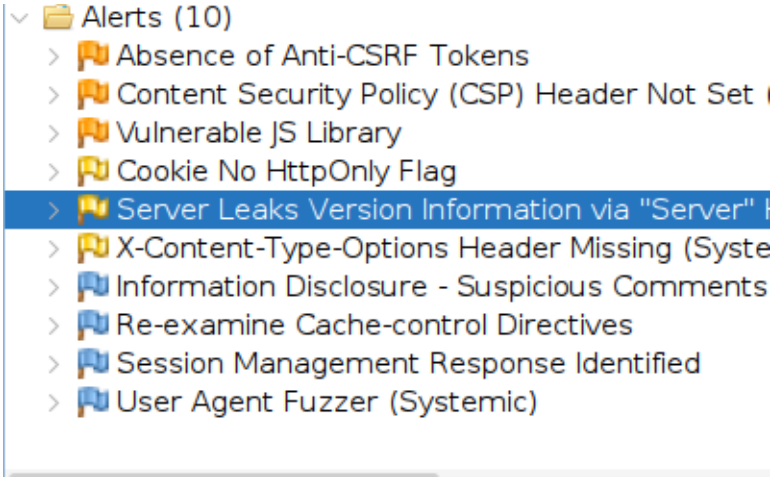
Auditoría de Vulnerabilidades Dinámicas (DAST) con OWASP ZAP

Ejecuté un análisis dinámico de seguridad de aplicaciones (DAST) utilizando **OWASP ZAP 2.17.0** sobre la dirección IP del entorno (`https://192.168.1.138`).

Resumen de Hallazgos:

- **Riesgo Alto / Crítico:** 0

- **Riesgo Medio:** 3 (Recomendación de política `Content-Security-Policy`, biblioteca estática Bootstrap en la interfaz navegable de DRF, manejo de tokens CSRF).
- **Riesgo Bajo:** 3 (Exposición de versión en el header `Server: nginx/1.31.6`, flags de cookies).
- **Informacionales:** 4



Conclusión del Análisis:

El proceso de bastionado básico implementado en **404Shop** elevó de manera sustancial la postura de seguridad del proyecto. La combinación de **cifrado en tránsito (TLS)**, **aislamiento de contenedores en redes privadas**, **supresión de datos de depuración** y **sanitización de respuestas JSON (DTOs)** garantiza una línea base sólida para entornos de staging o producción.

Pasos a seguir en el futuro

- **Ocultamiento de Versión del Servidor Web:** Agregar la directiva `server_tokens off;` en la configuración de Nginx para suprimir la cadena `nginx/1.31.6`.
- **Implementación de CSP (`Content-Security-Policy`):** Definir políticas estrictas en Nginx para mitigar riesgos de inyección de scripts externos.
- **Certificados Válidos (Producción):** Sustituir el certificado autofirmado por uno emitido por Let's Encrypt o una CA reconocida mediante certbot.

- **Protección de Tasa de Peticiones (Rate Limiting):** Configurar límites de peticiones en Nginx/Django REST Framework para mitigar fuerza bruta e intentos de denegación de servicio (DoS).

Nota final

Sobre este proyecto: Este laboratorio de bastionado (Hardening) fue desarrollado como parte de mi proceso de Aprendizaje autodidacta y especializacion práctica en ciberseguridad, analisis de seguridad y arquitectura de infraestructura

Como estudiante de ingeniería, reconozco que el mundo de aseguramiento de sistemas es amplio y constante evolución. Las configuraciones y controles implementados en esta primera fase representan una linea base de seguridad y un ejercicio practico de respuesta a vulnerabilidades comunes. Soy consciente de que no se trata de un bastionado empresarial perfecto, y que siempre existen capas adicionales de seguridad por explorar (COmo WAFs, gestión avanzada de secretos, políticas CSP complejas o automatización de auditorías CI/CD)

Mi objetivo con este proyecto no es solo aplicar teoría sino demostrar capacidad técnica para identificar fallos, investigar causas raíces, aplicar mitigacione efectivas y documentar el proceso bajo estándares profesionales

Creditos y agradecimientos: Tambien quiero expresar mi reconocimiento y agradecimiento al desarrollador original de la aplicacion base **404Shop,** aunque no tengo el gusto de conocerlo personalmente, la arquitectura full-stack desacoplada Django REST Framework + React + PostgreSQL que construyó está excelente y me sirvió mucho como escenario de pruebas perfecto para llevar a cabo este ejercicio de pentesting y hardening

Contacto y oportunidades profesionales

Actualmente me encuentro profundizando mis conocmientos en Analisis de seguridad / SOC, DevSecOps y seguridad en infraestructura Linux/Docker

Si eres un **reclutador,** líder técnico o empresa en búsqueda de un perfil apasionado por la ciberseguridad, autodidacta, meticuloso con la documentación y con un enfoque práctico en la resolución de problemas, estaré encantado de conectar:

Github academico: https://github.com/WashOrt26

LinkedIn: https://www.linkedin.com/in/michael-washington-ortega-hernández-0694492ba/

Correo Electrónico: maikotg55@gmail.com

Toda retroalimentación, sugerencia de mejora o crítica constructiva sobre este proyecto es más que bienvenida.